

Université Hassan II
FACULTÉ DES SCIENCES BEN MSICK
DÉPARTEMENT DE MATHÉMATIQUE ET INFORMATIQUE
- CASABLANCA -

Master : **DATA SCIENCES & BIG DATA**

PROJET DEVOPS

Mise en place d'une infrastructure DevOps locale avec CI/CD, Kubernetes, et Monitoring

Réalisé par :

Anouar MOHAMMED

Achraf CHAHID

Mohammed Khalil ABOUSSAAD

Sous l'encadrement de :

Prof. Benabbou FOUZIA

Table des matières

1	Introduction	2
1.1	Contexte du projets	2
1.2	Objectifs du projet	2
2	Cahier des Charges	4
2.1	Technologies utilisées	4
2.2	Architecture de l'Infrastructure DevOps	5
2.2.1	Introduction :	5
2.2.2	Diagramme d'architecture	5
2.2.3	Description du Diagramme :	6
2.2.4	Fonctionnement Global	6
2.2.5	Avantages de cette Architecture	7
3	Étapes Détaillées du Projet	8
3.1	Préparation de l'Environnement	8
3.2	Provisionnement de l'Infrastructure avec Terraform	8
3.2.1	Initialisation et Application de Terraform	13
3.2.2	Résultats du Provisionnement de l'Infrastructure	13
3.3	Configuration des machines avec Ansible	14
3.3.1	Conclusion	18
3.3.2	Description de l'Application	19
3.4	Configurer Jenkins sur la machine master	20
3.4.1	Tout d'abord en doit installer java après installer Jenkins :	20
3.4.2	En termine la configuration de Jenkins et l'installation des plugins	20
3.4.3	Après, il faut créer un nouveau pipeline :	20
3.5	Création du JenkinsFile	24
3.6	Tester le pipeline	29
3.7	Vérifier l'état et accéder à l'application	30

Chapitre 1

Introduction

1.1 Contexte du projets

L'infrastructure DevOps est devenue un élément clé dans le développement moderne des applications et la gestion des systèmes. Elle permet d'automatiser l'ensemble des processus de développement, de test, de déploiement et de gestion des infrastructures, tout en favorisant la collaboration entre les équipes de développement et d'opérations. L'objectif principal du DevOps est de réduire le temps nécessaire pour déployer de nouvelles versions d'applications tout en garantissant une qualité optimale et une gestion efficace des ressources.

Dans ce contexte, les entreprises cherchent à adopter des outils et des pratiques qui permettent d'améliorer leur réactivité et leur capacité à évoluer. Azure, une plateforme de cloud computing proposée par Microsoft, offre un ensemble de services robustes pour héberger des applications et gérer l'infrastructure. Le projet présenté ici vise à mettre en place une infrastructure DevOps complète sur Azure, en utilisant plusieurs outils clés tels que Terraform, Ansible, Kubernetes, Docker, et Jenkins.

- **Terraform** permet de définir l'infrastructure sous forme de code (IaC - Infrastructure as Code), facilitant ainsi la gestion et la mise à jour des ressources sur Azure.
- **Ansible** est un outil d'automatisation des configurations, qui nous permet d'installer et de configurer les ressources sur les machines virtuelles de manière déclarative.
- **Kubernetes** est une plateforme d'orchestration de conteneurs qui permet de déployer, gérer et scaler les applications de manière efficace.
- **Docker** est utilisé pour la conteneurisation de l'application, ce qui permet de rendre l'application portable et facile à déployer dans différents environnements.
- **Jenkins** est un outil d'intégration continue et de déploiement continu (CI/CD) qui va automatiser le pipeline de développement, du code source à la mise en production.

L'intégration de ces outils permet d'automatiser les processus, de réduire les erreurs humaines, de maintenir un haut niveau de disponibilité, et de faciliter la gestion de l'infrastructure tout en assurant une évolutivité maximale.

1.2 Objectifs du projet

Le principal objectif de ce projet est de mettre en place une infrastructure DevOps automatisée sur Azure, en utilisant les technologies modernes pour gérer un cluster Kubernetes, déployer des applications conteneurisées et superviser l'environnement avec des outils de monitoring. Les objectifs spécifiques sont les suivants :

- **Automatiser la création d'un cluster Kubernetes sur Azure :**
 - Utilisation de Terraform pour provisionner les ressources nécessaires sur Azure, y compris les machines virtuelles et le cluster Kubernetes.
 - Configuration des nœuds du cluster et intégration avec l'outil d'orchestration Kubernetes.
- **Déployer une application conteneurisée avec Docker :**
 - Développer une application simple (par exemple, une API Flask ou Node.js) et la conteneuriser en utilisant Docker.
 - Pousser l'image Docker sur Docker Hub pour la rendre accessible pour le déploiement sur Kubernetes.
- **Mettre en place une chaîne CI/CD avec Jenkins :**
 - Intégrer Jenkins pour automatiser le pipeline de développement et de déploiement.
 - Configurer un pipeline qui effectue les étapes suivantes : récupération du code source, construction de l'image Docker, push de l'image vers Docker Hub, et déploiement sur le cluster Kubernetes.

Ce projet met l'accent sur l'automatisation, la scalabilité, et la résilience de l'infrastructure. L'objectif final est de démontrer l'efficacité des pratiques DevOps sur le cloud avec Azure, et de fournir une solution prête à être utilisée en production pour le déploiement et la gestion d'applications conteneurisées dans un environnement Kubernetes.

Chapitre 2

Cahier des Charges

2.1 Technologies utilisées

Le projet repose sur l'utilisation de plusieurs outils et services afin de mettre en place une infrastructure DevOps complète. Chaque composant de l'infrastructure a été choisi en fonction de sa capacité à automatiser et gérer les processus tout en garantissant une scalabilité et une efficacité optimale.

Voici les technologies clés utilisées dans ce projet :

1. Infrastructure et Orchestration

- **Cloud Provider : Azure** – Utilisé pour héberger l'infrastructure, y compris les machines virtuelles qui constituent le cluster Kubernetes.
- **Kubernetes** – Orchestration des conteneurs pour assurer la **scalabilité, la gestion et l'automatisation** du déploiement des applications.
- **Cluster Kubernetes** – Composé d'un **nœud master** et d'un ou plusieurs **nœuds workers**, provisionné avec **Terraform** et configuré avec **Ansible**.

2. Conteneurisation et Déploiement

- **Docker** – Utilisé pour **conteneuriser l'application** en l'emballant avec toutes ses dépendances. Cela garantit sa portabilité et simplifie son déploiement sur Kubernetes.
- **Docker Hub** – Registre distant utilisé pour stocker et gérer les images Docker avant leur déploiement sur le cluster.

3. Automatisation de l'Infrastructure

- **Terraform** – Outil d'Infrastructure as Code (IaC) permettant de provisionner les ressources sur Azure, notamment les machines virtuelles pour le cluster Kubernetes.
- **Ansible** – Utilisé pour automatiser la configuration des machines virtuelles, installer Docker, Kubernetes (kubectl, kubeadm) et assurer une configuration homogène du cluster.

4. CI/CD et Intégration Continue

- **Jenkins** – Serveur d'intégration continue (CI) et de déploiement continu (CD) utilisé pour **automatiser l'intégralité du pipeline**.

5. Gestion du Code Source et Automatisation

- **GitHub** – Utilisé comme gestionnaire de version pour le code source. Un **webhook** est configuré pour déclencher automatiquement le pipeline Jenkins après chaque modification.
- **Visual Studio Code** – Environnement de développement utilisé par les développeurs pour modifier et gérer le code de l'application.

2.2 Architecture de l'Infrastructure DevOps

2.2.1 Introduction :

L'architecture mise en place repose sur une approche DevOps intégrant l'automatisation de l'infrastructure, la gestion des configurations et le déploiement continu. Elle combine plusieurs outils pour assurer une gestion efficace du cluster Kubernetes sur Azure, tout en garantissant une scalabilité et une maintenance simplifiées.

2.2.2 Diagramme d'architecture

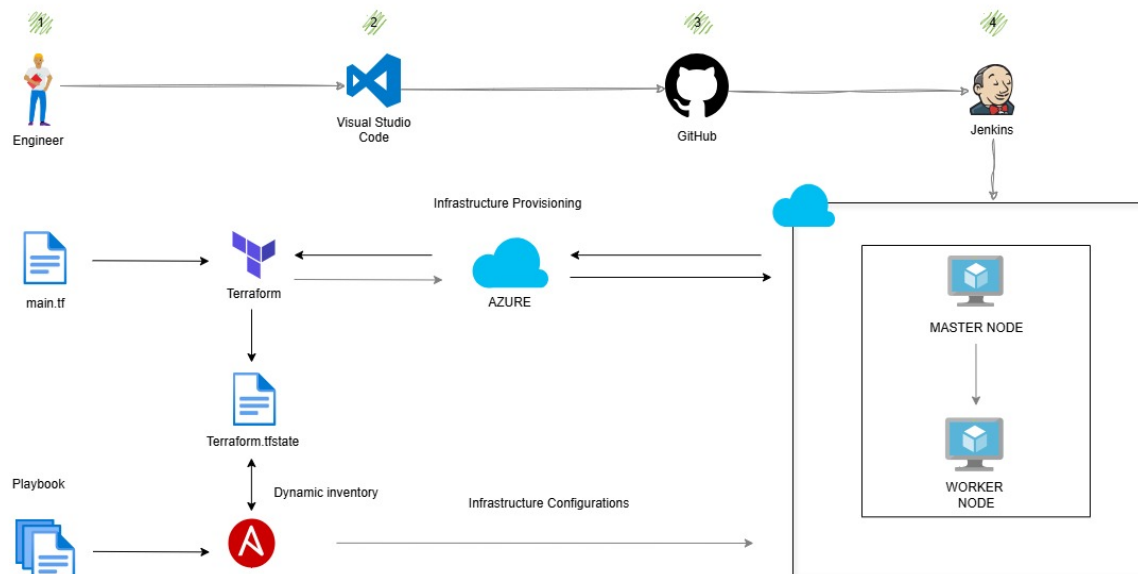


FIGURE 2.1 – Diagramme d'architecture du Projet

2.2.3 Description du Diagramme :

Le diagramme d'architecture de l'infrastructure DevOps illustre les composants clés et leur interaction pour mettre en place un environnement de développement automatisé et une gestion d'infrastructure à l'aide d'outils modernes. Voici une description de chaque élément de l'architecture et de leurs relations :

- **Développement et Gestion du Code :**

- **Développeur** : Responsable du développement de l'application et de la gestion du code source.
- **Visual Studio Code** : Environnement de développement utilisé pour écrire et gérer l'application ainsi que les fichiers de configuration **Terraform** et **Ansible**.
- **GitHub** : S Stockage centralisé du code source et des fichiers de configuration, avec un **webhook** configuré pour déclencher automatiquement les pipelines CI/CD.

- **Provisioning de l'Infrastructure :**

- **Terraform** : Utilisé pour la gestion de l'infrastructure en tant que code (Infrastructure as Code - IaC). Il permet de provisionner les ressources cloud sur **Azure**, notamment :
 - Une machine virtuelle pour le **Master Node**
 - Une machine virtuelle pour le **Worker Node**
- Le fichier **main.tf** définit les ressources à provisionner, et l'état du déploiement est stocké dans **terraform.tfstate**.

- **Configuration des Machines Virtuelles**

- **Ansible** : Responsable de l'installation et de la configuration des services sur les machines virtuelles provisionnées. Il utilise :
 - Un **Playbook** pour définir les actions d'installation et de configuration (exemple : installation de Docker, Kubernetes, kubectl, kubeadm).
 - Un **inventory** dynamique basé sur l'état Terraform (**terraform.tfstate**) pour cibler les machines à configurer.

- **Intégration et Déploiement Continu (CI/CD) avec Webhook :**

- **GitHub Webhook** : Déclenche automatiquement un pipeline Jenkins à chaque push ou modification du code source.
- **Jenkins** : Orchestration des tâches DevOps, y compris :
 - Récupération du code de l'application depuis GitHub.
 - Exécution des pipelines CI/CD pour **tester**, **dockeriser** et **déployer** l'application sur le cluster Kubernetes.

2.2.4 Fonctionnement Global

- Le **développeur** écrit le code de l'application dans **VS Code** et le versionne sur **GitHub**.
- Un **webhook GitHub** détecte le changement et **déclenche automatiquement** un pipeline Jenkins.

- **Terraform** est utilisé pour provisionner l'infrastructure sur **Azure** (machines virtuelles pour le cluster Kubernetes).
- **Ansible** configure ces machines en installant les outils nécessaires pour Kubernetes.
- **Jenkins** récupère le code source de l'application et exécute les pipelines CI/CD pour tester et déployer l'application.
- L'application est déployée sur le **cluster Kubernetes** (Master et Worker Nodes).

2.2.5 Avantages de cette Architecture

- **Automatisation complète** du provisioning et de la configuration de l'infrastructure.
- **Déclenchement instantané** du pipeline CI/CD sans intervention manuelle
- **Scalabilité et flexibilité** grâce à l'utilisation d'Azure et de Kubernetes.
- **Séparation des responsabilités** entre le développement de l'application, l'infrastructure et le CI/CD.
- **Réduction des erreurs manuelles** et gain de temps grâce à l'IaC et l'automatisation.
- **Réactivité améliorée** : chaque modification du code est testée et déployée immédiatement.

Chapitre 3

Étapes Détaillées du Projet

3.1 Préparation de l'Environnement

- Créer un compte Azure.
- Créer un compte Docker.
- Activer WSL (Windows Subsystem for Linux) ou utiliser une machine virtuelle (MV).
- Installer Terraform et Ansible dans WSL ou la machine virtuelle.

3.2 Provisionnement de l'Infrastructure avec Terraform

```
1  # Create Resource Group
2  resource "azurerm_resource_group" "rg" {
3      name      = "myResourceGroup"
4      location = "West Europe"
5
6  }
7
8  # Create Virtual Network
9  resource "azurerm_virtual_network" "vnet" {
10     name            = "myVNet"
11     location        = azurerm_resource_group.rg.location
12     resource_group_name = azurerm_resource_group.rg.name
13     address_space    = ["10.0.0.0/16"]
14 }
15
16 # Create Subnet
17 resource "azurerm_subnet" "subnet" {
18     name            = "mySubnet"
19     resource_group_name = azurerm_resource_group.rg.name
20     virtual_network_name = azurerm_virtual_network.vnet.name
21     address_prefixes   = ["10.0.1.0/24"]
22 }
```

- **1. Création du Groupe de Ressources** Un Resource Group est une entité logique qui permet de regrouper et gérer les ressources Azure associées à un projet spécifique. Dans cette étape, un groupe de ressources nommé "myResourceGroup" est créé dans la région West Europe, garantissant ainsi une centralisation et une gestion efficace des ressources de l'infrastructure.
- **2. Mise en Place du Réseau Virtuel (VNet)** Le Virtual Network (VNet) est utilisé pour définir un espace d'adressage IP privé sur Azure. Ici, le réseau "myVNet" est créé avec une plage d'adresses 10.0.0.0/16, permettant d'héberger plusieurs sous-réseaux et d'assurer la connectivité entre les ressources du même réseau.
- **3. Création du Sous-Réseau (Subnet)** Un sous-réseau ("mySubnet") est défini à l'intérieur du VNet avec une plage d'adresses 10.0.1.0/24. Ce sous-réseau permet d'organiser le réseau et d'héberger les machines virtuelles tout en contrôlant leur accès via des règles de sécurité.

```
1
2  # Create Network Security Group (NSG)
3  resource "azurerm_network_security_group" "nsg" {
4      name                = "myNSG"
5      location            = azurerm_resource_group.rg.location
6      resource_group_name = azurerm_resource_group.rg.name
7  }
8
9  # Add NSG Rule for SSH
10 resource "azurerm_network_security_rule" "ssh" {
11     name                = "AllowSSH"
12     priority            = 100
13     direction          = "Inbound"
14     access              = "Allow"
15     protocol            = "Tcp"
16     source_port_range   = "*"
17     destination_port_range = "22"
18     source_address_prefix = "0.0.0.0/0"
19     destination_address_prefix = "*"
20     resource_group_name  = azurerm_resource_group.rg.name
21     network_security_group_name = azurerm_network_security_group.nsg.name
22 }
23
24 # Associate NSG with Subnet
25 resource
26 "azurerm_subnet_network_security_group_association" "nsg_association" {
27     subnet_id                = azurerm_subnet.subnet.id
28     network_security_group_id = azurerm_network_security_group.nsg.id
29 }
```

- **4. Définition d'un Groupe de Sécurité Réseau (NSG)** Un Network Security Group (NSG) est créé pour filtrer le trafic réseau entrant et sortant au niveau du sous-réseau. Le "myNSG" servira à appliquer des règles de sécurité pour contrôler les connexions aux ressources.u.
- **5. Configuration d'une Règle de Sécurité pour SSH** Un sous-réseau ("my-Subnet") est défini à l'intérieur du VNet avec une plage d'adresses 10.0.1.0/24. Ce sous-réseau permet d'organiser le réseau et d'héberger les machines virtuelles tout en contrôlant leur accès via des règles de sécurité.
- **6. Association du NSG avec le Sous-Réseau** Le NSG est lié au sous-réseau, ce qui signifie que toutes les ressources connectées à ce sous-réseau seront soumises aux règles de sécurité définies, renforçant ainsi la protection du réseau.

```
1  # Create Public IP for Master Node
2  resource "azurerm_public_ip" "master_ip" {
3      name                = "master-public-ip"
4      location            = azurerm_resource_group.rg.location
5      resource_group_name = azurerm_resource_group.rg.name
6      allocation_method    = "Static"
7      sku                  = "Standard"
8  }
9
10 # Create Public IP for Worker Node 1
11 resource "azurerm_public_ip" "worker1_ip" {
12     name                = "worker1-public-ip"
13     location            = azurerm_resource_group.rg.location
14     resource_group_name = azurerm_resource_group.rg.name
15     allocation_method    = "Static"
16     sku                  = "Standard"
17 }
```

- **7. Création d'Adresses IP Publiques pour les Machines Virtuelles** Deux adresses IP publiques statiques sont définies pour permettre l'accès externe :
 - "master-public-ip" : pour la machine principale (Master Node).
 - "worker1-public-ip" : pour la première machine de travail (Worker Node 1).

```
1
2 # Create Network Interface for Master Node
3 resource "azurerm_network_interface" "master_nic" {
4     name                = "master-nic"
5     location            = azurerm_resource_group.rg.location
6     resource_group_name = azurerm_resource_group.rg.name
7
8     ip_configuration {
9         name                = "internal"
10        subnet_id           = azurerm_subnet.subnet.id
11        private_ip_address_allocation = "Dynamic"
12        public_ip_address_id = azurerm_public_ip.master_ip.id
13    }
14 }
15
16 # Create Network Interface for Worker Node 1
17 resource "azurerm_network_interface" "worker1_nic" {
18     name                = "worker1-nic"
19     location            = azurerm_resource_group.rg.location
20     resource_group_name = azurerm_resource_group.rg.name
21
22     ip_configuration {
23         name                = "internal"
24         subnet_id           = azurerm_subnet.subnet.id
25         private_ip_address_allocation = "Dynamic"
26         public_ip_address_id = azurerm_public_ip.worker1_ip.id
27     }
28 }
```

- **8. Configuration des Interfaces Réseau (NICs)** Chaque machine virtuelle nécessite une interface réseau (Network Interface - NIC) pour être connectée au sous-réseau :
 - **"master-nic"** : pour la machine Master, associée à l'adresse IP publique du Master.
 - **"worker1-nic"** : pour le Worker 1, associée à son adresse IP publique.
- Ces interfaces assurent la communication entre les machines et le réseau Azure

```
1  # Create Master VM
2  resource "azurerm_linux_virtual_machine" "master_vm" {
3      name                = "masterVM"
4      resource_group_name = azurerm_resource_group.rg.name
5      location            = azurerm_resource_group.rg.location
6      size                = "Standard_B2ms"
7      admin_username      = "azureuser"
8      network_interface_ids = [azurerm_network_interface.master_nic.id]
9      admin_ssh_key {
10         username    = "azureuser"
11         public_key  = file("/home/khalil/.ssh/id_rsa.pub")
12     }
13     os_disk {
14         caching              = "ReadWrite"
15         storage_account_type = "Standard_LRS"
16     }
17     source_image_reference {
18         publisher = "Canonical"
19         offer     = "UbuntuServer"
20         sku       = "18.04-LTS"
21         version   = "latest"
22     }
23 }
24 # Create Worker VM 1
25 resource "azurerm_linux_virtual_machine" "worker1_vm" {
26     name                = "worker1VM"
27     resource_group_name = azurerm_resource_group.rg.name
28     location            = azurerm_resource_group.rg.location
29     size                = "Standard_B2s"
30     admin_username      = "azureuser"
31     network_interface_ids = [azurerm_network_interface.worker1_nic.id]
32     admin_ssh_key {
33         username    = "azureuser"
34         public_key  = file("/home/khalil/.ssh/id_rsa.pub")
35     }
36     os_disk {
37         caching              = "ReadWrite"
38         storage_account_type = "Standard_LRS"
39     }
40     source_image_reference {
41         publisher = "Canonical"
42         offer     = "UbuntuServer"
43         sku       = "18.04-LTS"
44         version   = "latest"
45     }
46 }
```

- **9. Déploiement de la Machine Virtuelle Master** Une machine virtuelle Linux ("masterVM") est créée avec les spécifications suivantes :
 - Image Ubuntu 18.04 LTS.
 - Taille Standard_B2ms adaptée aux charges de travail moyennes.
 - Authentification via clé SSH (stockée dans /home/khalil/.ssh/id_rsa.pub).
 - Disque principal de stockage de type Standard_LRS.Cette machine servira de nœud principal pour gérer les opérations et orchestrer les services dans l'environnement cloud.
- **10. Déploiement de la Machine Virtuelle Worker 1** Une seconde machine virtuelle ("worker1VM") est déployée avec une configuration similaire à celle du Master, mais avec une taille de machine légèrement inférieure Standard_B2s. Cette machine servira de nœud de travail dans l'infrastructure.

3.2.1 Initialisation et Application de Terraform

Avant de provisionner l'infrastructure, Terraform doit être initialisé afin de télécharger les plugins nécessaires et configurer le backend. Pour cela, on commence par exécuter la commande suivante dans le répertoire contenant le fichier `main.tf` :

- `terraform init`

Cette commande télécharge les fournisseurs nécessaires et prépare l'environnement Terraform. Ensuite, pour vérifier la configuration et simuler les changements sans les appliquer, on utilise la commande suivante :

- `terraform plan`

Elle permet d'obtenir un aperçu des ressources qui seront créées ou modifiées. Une fois la configuration validée, on applique les changements avec la commande suivante :

- `terraform apply -auto-approve`

Cette commande crée et configure automatiquement toutes les ressources définies dans `main.tf`.

3.2.2 Résultats du Provisionnement de l'Infrastructure

Après l'exécution réussie du script Terraform, les ressources ont été provisionnées avec succès sur Microsoft Azure sous l'abonnement "Azure for Students". Comme illustré dans

Accueil >

Toutes les ressources ✨ ...

Répertoire par défaut (achrafchahid3etuetuunivh2c.onmicrosoft.com)

+ Créer | Gérer la vue | Actualiser | Exporter au format CSV | Ouvrir une requête | Attribuer des étiquettes | Supprimer

Filter un champ... | Abonnement égal à tout | Groupe de ressources égal à tout | Type égal à tout | Emplacement égal à tout | Ajouter un filtre

3 Ressources non sécurisées | 14 Recommandations | 0 Ressources modifiées | Aucun regroupement | Vue liste

Nom ↑↓	Type ↑↓	Groupe de ressources ↑↓	Emplacement ↑↓	Abonnement ↑↓	
masterVM	Machine virtuelle	myResourceGroup	West Europe	Azure for Students	...
masterVM_OsDisk_1_45e06e07d2da430ebd6662a416ddb85	Disque	MYRESOURCEGROUP	West Europe	Azure for Students	...
myNSG	Groupe de sécurité réseau	myResourceGroup	West Europe	Azure for Students	...
myVNet	Réseau virtuel	myResourceGroup	West Europe	Azure for Students	...
NetworkWatcher_eastus	Network Watcher	NetworkWatcherRG	East US	Azure for Students	...
NetworkWatcher_westeurope	Network Watcher	NetworkWatcherRG	West Europe	Azure for Students	...
worker1-nic	Interface réseau	myResourceGroup	West Europe	Azure for Students	...
worker1-public-ip	Adresse IP publique	myResourceGroup	West Europe	Azure for Students	...
worker1VM	Machine virtuelle	myResourceGroup	West Europe	Azure for Students	...
worker1VM_OsDisk_1_e05b26ce27df4941b33a610f47a5047a	Disque	MYRESOURCEGROUP	West Europe	Azure for Students	...

FIGURE 3.1 – Résultats du Provisionnement de l'Infrastructure

3.3 Configuration des machines avec Ansible

```

1  name: Install Docker Engine and k3s on master and worker nodes
2  hosts: all
3  become: yes
4  tasks:
5
6  - name: Update apt cache
7    apt:
8      update_cache: yes
9
10 - name: Install dependencies
11   apt:
12     name:
13       - apt-transport-https
14       - ca-certificates
15       - curl
16       - gnupg
17       - software-properties-common
18     state: present
19
20 - name: Remove old Kubernetes repository if it exists
21   file:
22     path: /etc/apt/sources.list.d/kubernetes.list
23     state: absent
24
25 - name: Add Kubernetes GPG key
26   apt_key:
27     url: https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key
28     state: present

```

```
1  - name: Add Kubernetes apt repository
2    apt_repository:
3      repo: "deb https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /"
4      state: present
5      filename: "kubernetes"
6
7  - name: Add Docker's official GPG key
8    apt_key:
9      url: https://download.docker.com/linux/ubuntu/gpg
10     keyring: /usr/share/keyrings/docker-archive-keyring.gpg
11     state: present
12
13  - name: Add Docker's stable repository
14    apt_repository:
15      repo: "
16        deb [arch=amd64 signed-by=/usr/share/keyrings/
17        docker-archive-keyring.gpg] https://download.docker.com/
18        linux/ubuntu {{ ansible_distribution_release }} stable
19      "
20      state: present
21      filename: "docker"
22
23  - name: Update apt cache after adding repositories
24    apt:
25      update_cache: yes
26
27  - name: Install Docker Engine
28    apt:
29      name:
30        - docker-ce
31        - docker-ce-cli
32        - containerd.io
33      state: present
34
35  - name: Ensure Docker is running and enabled on boot
36    service:
37      name: docker
38      state: started
39      enabled: yes
40
41  - name: Add azureuser to docker group
42    user:
43      name: azureuser
44      groups: docker
45      append: yes
```

```
1  - name: Install k3s on master node
2    shell: |
3      curl -sL https://get.k3s.io | sh -
4    when: inventory_hostname in groups['masters']
5  - name: Wait for k3s server to be ready
6    wait_for:
7      port: 6443
8      host: "{{ ansible_default_ipv4.address }}"
9      delay: 10
10     timeout: 60
11    when: inventory_hostname in groups['masters']
12  - name: Get k3s join token from master node
13    shell: "cat /var/lib/rancher/k3s/server/node-token"
14    register: k3s_token
15    delegate_to: "{{ groups['masters'][0] }}"
16    run_once: true
17    changed_when: false
18  - name: Install k3s on worker nodes
19    shell: |
20      curl -sL https://get.k3s.io | K3S_URL=https://{{
21        hostvars[groups['masters']
22          [0]].ansible_default_ipv4.address }}:6443 K3S_TOKEN={{
23        k3s_token.stdout }} sh -
24    when: inventory_hostname in groups['workers']
25  - name: Get latest stable kubectl version
26    shell: "curl -L -s https://dl.k8s.io/release/stable.txt"
27    register: kubectl_version
28    changed_when: false
29  - name: Install kubectl
30    get_url:
31      url: "
32        https://dl.k8s.io/release/{{ kubectl_version.stdout }}/
33        bin/linux/amd64/kubectl
34      "
35      dest: "/usr/local/bin/kubectl"
36      mode: '0755'
37  - name: Configure kubectl for k3s
38    shell: |
39      mkdir -p /home/azureuser/.kube
40      if [ -f /etc/rancher/k3s/k3s.yaml ]; then
41        sudo cp /etc/rancher/k3s/k3s.yaml /home/azureuser/.kube/config
42        sudo chown azureuser:azureuser /home/azureuser/.kube/config
43      fi
44    when: inventory_hostname in groups['masters']
```

Ce playbook Ansible permet d'automatiser le déploiement d'un cluster Kubernetes léger en installant et configurant Docker ainsi que K3s sur un ensemble de nœuds. Il est conçu pour garantir une installation fluide et uniforme en éliminant les étapes manuelles, ce qui est essentiel pour assurer la cohérence et la reproductibilité du processus.

- **Mise à jour et installation des dépendances :** Le processus commence par la mise à jour du cache des paquets afin de s'assurer que les dernières versions disponibles sont utilisées. Ensuite, plusieurs dépendances essentielles sont installées, notamment apt-transport-https, ca-certificates, curl, gnupg et software-properties-common. Ces paquets sont indispensables pour la gestion des dépôts et l'authentification des clés GPG, garantissant ainsi une installation sécurisée des logiciels nécessaires.
- **Configuration des dépôts pour Kubernetes et Docker :** Avant d'installer Docker et Kubernetes, il est nécessaire d'ajouter leurs dépôts officiels. Le playbook commence par supprimer d'éventuels anciens fichiers de configuration des dépôts Kubernetes afin d'éviter tout conflit avec des versions précédemment installées. Ensuite, il ajoute la clé GPG de Kubernetes et configure le dépôt officiel de Kubernetes pour s'assurer que les packages seront téléchargés depuis une source fiable et mise à jour.

De même, pour Docker, le playbook ajoute la clé GPG officielle et configure le dépôt stable de Docker en fonction de la distribution Linux utilisée. Cette étape permet d'obtenir et d'installer la dernière version stable des composants Docker.
- **Installation et configuration de Docker :** Une fois les dépôts configurés, Docker Engine est installé, accompagné de ses composants principaux docker-ce, docker-ce-cli et containerd.io. Le service Docker est ensuite démarré et configuré pour se lancer automatiquement à chaque redémarrage du système, garantissant ainsi sa disponibilité en permanence.

Par ailleurs, pour éviter d'avoir à utiliser des commandes en mode super-utilisateur, l'utilisateur azureuser est ajouté au groupe docker. Cela lui permet d'exécuter des commandes Docker sans avoir besoin de préfixer chaque commande avec sudo, ce qui facilite l'utilisation et la gestion des conteneurs.
- **Installation et configuration de K3s sur le nœud master :** Sur le nœud désigné comme master, K3s est installé à l'aide d'un script officiel fourni par Rancher. Ce script télécharge et installe K3s en le configurant automatiquement en tant que serveur principal du cluster.

Après l'installation, un délai est introduit pour attendre la mise en place complète du serveur K3s et s'assurer qu'il est bien actif sur le port 6443, qui est utilisé pour la communication entre les composants Kubernetes. Cette attente est essentielle pour éviter que les nœuds worker ne tentent de se connecter avant que le serveur ne soit entièrement opérationnel.

Une fois le serveur prêt, le playbook récupère le jeton d'authentification de K3s, un élément clé permettant aux nœuds worker de rejoindre le cluster. Cette récupération est réalisée uniquement sur le nœud master et partagée avec les autres nœuds.
- **Installation et connexion des nœuds worker :** Sur les nœuds worker, K3s est installé en utilisant le même script d'installation que sur le master, mais avec une

configuration spécifique permettant de les connecter directement au cluster. Pour cela, l'URL du serveur K3s (situé sur le nœud master) et le jeton d'authentification récupéré précédemment sont utilisés. Cette étape permet aux nœuds worker de s'enregistrer auprès du cluster et de commencer à exécuter des charges de travail Kubernetes.

- **Installation de kubectl et configuration de l'accès au cluster :** Le playbook automatise également l'installation de kubectl, l'outil en ligne de commande permettant d'interagir avec un cluster Kubernetes. Pour garantir l'installation de la dernière version stable, le playbook récupère dynamiquement la version actuelle depuis le site officiel de Kubernetes avant de télécharger et d'installer l'exécutable. Enfin, pour faciliter l'administration du cluster, le fichier de configuration de Kubernetes (kubeconfig) est copié depuis le répertoire de K3s vers le répertoire personnel de l'utilisateur azureuser. Cette configuration permet à l'utilisateur d'exécuter directement des commandes kubectl pour gérer les ressources Kubernetes sans avoir à spécifier manuellement l'adresse du serveur ou les informations d'authentification.

3.3.1 Conclusion

Ce playbook Ansible offre une approche entièrement automatisée pour le déploiement d'un cluster Kubernetes avec Docker et K3s. Grâce à cette automatisation, l'installation est rapide, fiable et reproductible sur plusieurs machines. En supprimant les étapes manuelles et en garantissant une configuration cohérente, ce processus simplifie considérablement la mise en place d'un environnement Kubernetes opérationnel, facilitant ainsi le déploiement et la gestion des applications conteneurisées.

3.3.2 Description de l'Application

L'application utilisée dans ce projet est basée sur **Python Flask - Demo Web Application**, une application web légère développée avec **Flask**. Elle permet d'afficher en temps réel des informations système telles que l'utilisation du **CPU**, de la **mémoire**, des **entrées/sorties** et des processus en cours.

Cette application a été adaptée et intégrée dans notre infrastructure DevOps, où elle est déployée de manière automatisée via un pipeline Jenkins.

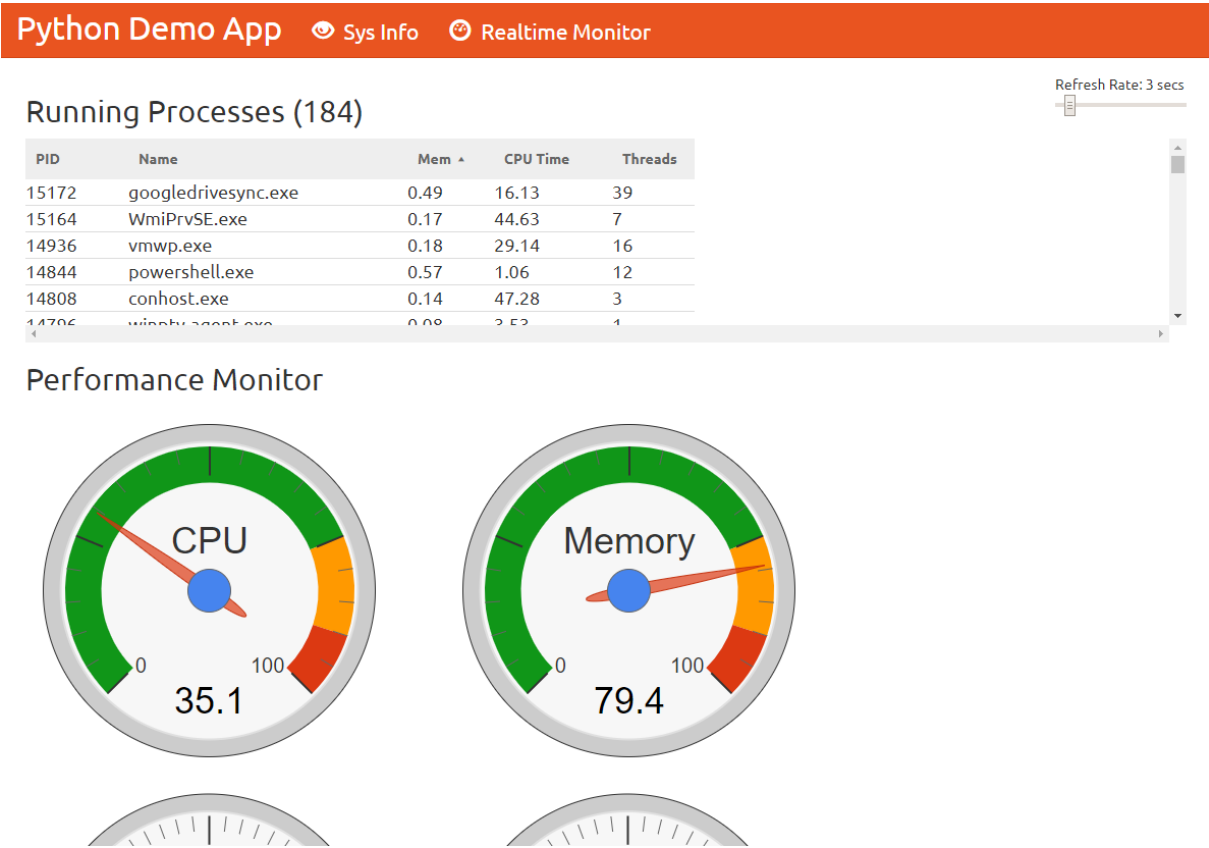


FIGURE 3.2

3.4 Configurer Jenkins sur la machine master

Jenkins est un serveur d'automatisation open source utilisé pour mettre en place des pipelines CI/CD (Intégration Continue / Déploiement Continu).

3.4.1 Tout d'abord en doit installer java après installer Jenkins :

- `sudo apt install openjdk-17-jdk -y`
- `sudo apt install jenkins -y`
- `sudo systemctl start jenkins`
- `sudo systemctl enable jenkins`

après on accède a jenkins depuis le port 8080 en utilisant l'IP du master VM :

- [http ://172.201.196.66 :8080/](http://172.201.196.66:8080/)

3.4.2 En termine la configuration de Jenkins et l'installation des plugins

Après on ajoute les identifiants pour docker, k3s et Github on presque les mêmes étapes

- Accédez à Jenkins > Manage Jenkins > Manage Credentials
- Cliquez sur Global credentials (unrestricted).
- Cliquez sur Add Credentials
- Kind : Username with password
- Scope : Global
- Username : Votre nom d'utilisateur Docker Hub
- Password : Votre mot de passe Docker Hub
- ID : dockerhub-credentials (ou un autre identifiant unique)
- Description : Credentials for Docker Hub

3.4.3 Après, il faut créer un nouveau pipeline :

- Sur le tableau de bord de Jenkins, cliquez sur "New Item".
- Donnez un nom à votre pipeline.
- Sélectionnez "Pipeline" comme type de projet.

New credentials

Kind: Username with password

Scope: Global (Jenkins, nodes, items, all child items, etc)

Username:

☐ Treat username as secret

Password:

Create

FIGURE 3.3 – Création de nouveaux identifiants

Credentials

T	P	Store	I	Domain	ID	Name
		System		(global)	569344dc-008f-46ed-816f-aa33f744ad6b	cchahid
		System		(global)	azure-jenkins-key	jenkins
		System		(global)	docker-hub-credentials	cchahi/*****
		System		(global)	azure-kubeconfig	kube.yaml

FIGURE 3.4 – La liste des identifiants

Pour que ce déclencheur se lance, il faut ajouter un webhook sur notre repository avec l'adresse IP du master (qui contient JENKINS) en spécifiant cette URL :

— <http://172.201.196.66:8080/github-webhook/>

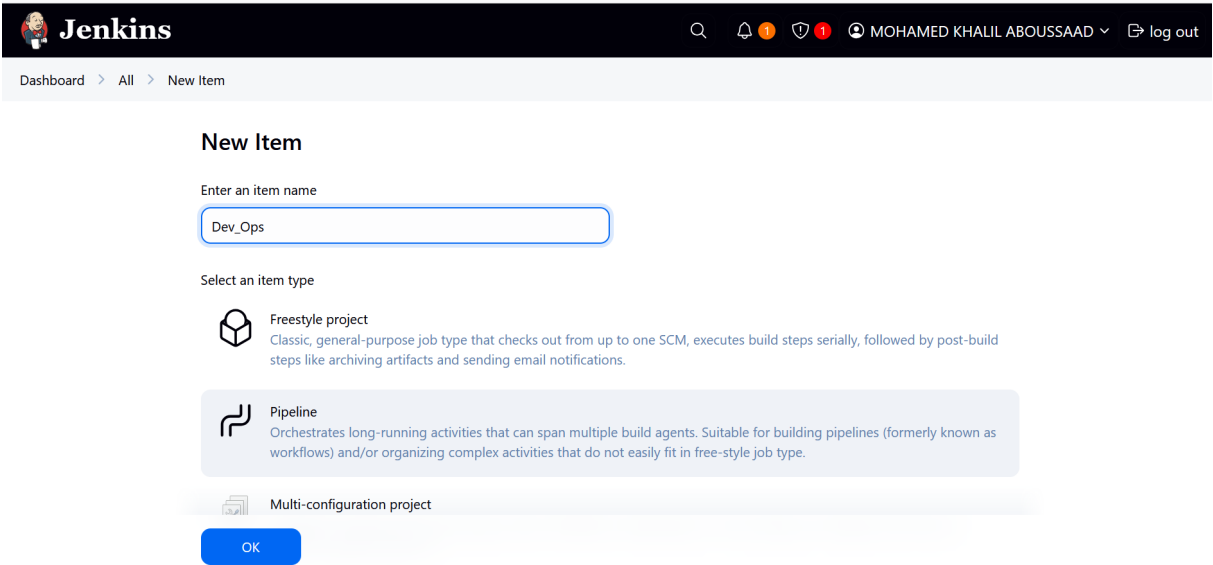


FIGURE 3.5 – Création d’un nouveau pipeline

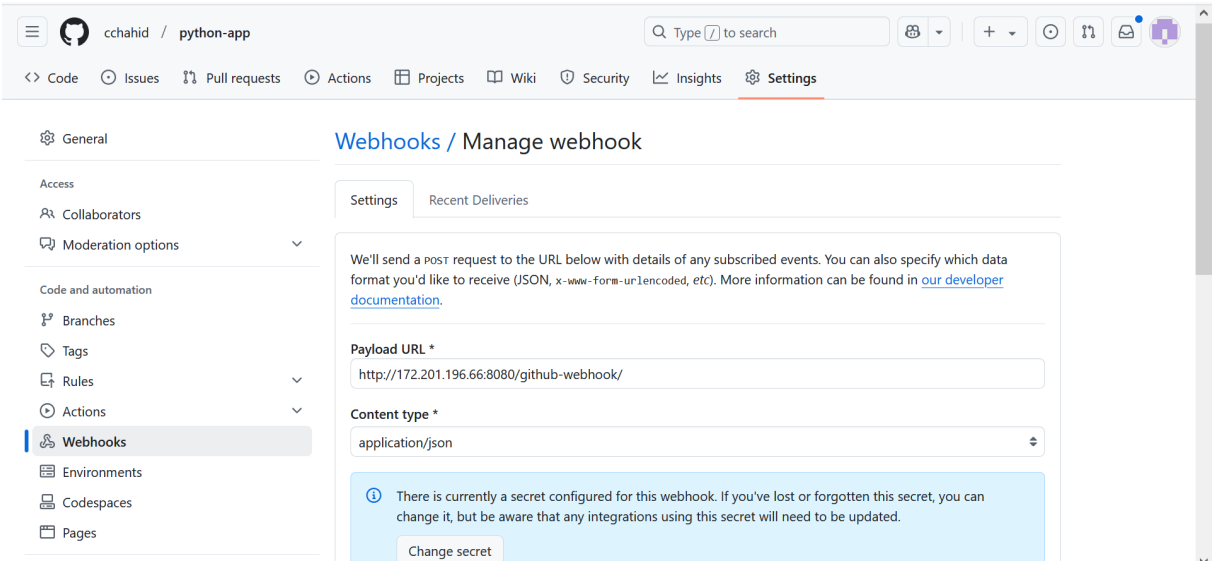


FIGURE 3.6 – Création d’un webhook

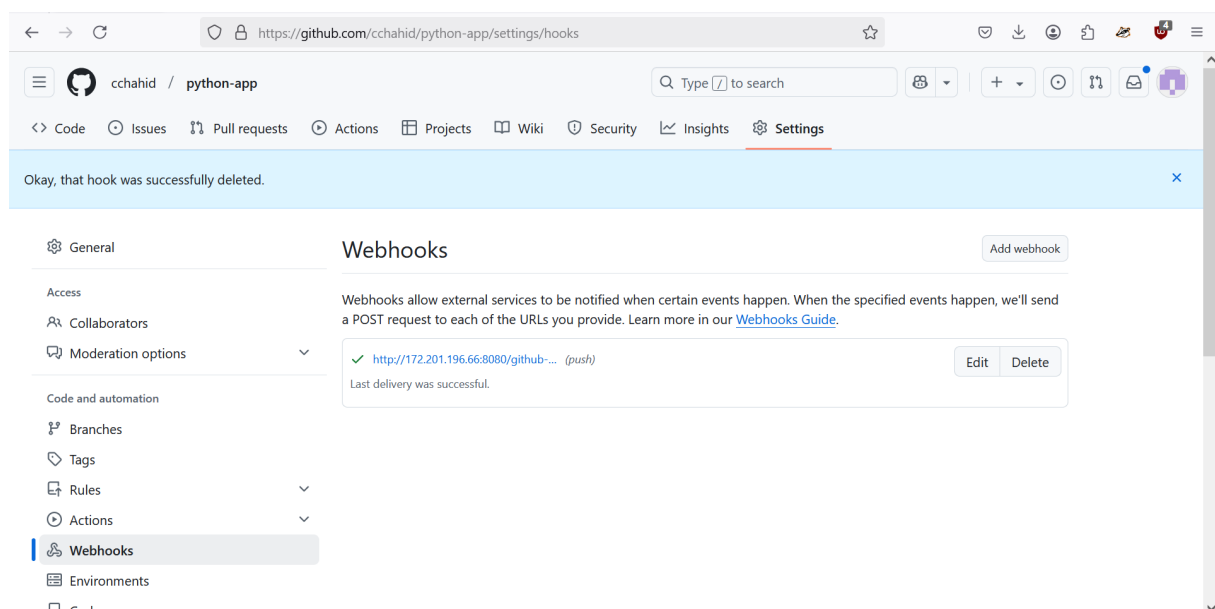


FIGURE 3.7 – L'état du webhook

3.5 Création du JenkinsFile

Le fichier Jenkinsfile est un script qui définit un pipeline Jenkins pour automatiser les étapes de build, de test et de déploiement d'une application qui permet de :

- Automatiser les processus
- Standardiser les étapes
- Faciliter la maintenance
- Intégrer des outils externes

```
1 pipeline {
2   agent any
3
4   environment {
5     KUBECONFIG = '/var/lib/jenkins/.kube/config'
6     IMAGE_NAME = 'python-app'
7     CONTAINER_PORT = '5000'
8     HOST_PORT = '5001'
9     DOCKER_REGISTRY = 'docker.io/cchahi'
10    DOCKER_TAG = 'latest.${BUILD_NUMBER}'
11    ANSIBLE_PLAYBOOK = 'deploy.yml'
12    SSH_KEY = '/var/lib/jenkins/.ssh/jen-ssh-key.txt'
13    K8S_MANIFEST_PATH = 'k8s'
14    DEPLOYMENT_NAME = 'python-app'
15    CONTAINER_NAME = 'python-app-container'
16  }
17
18  stages {
19    stage('Checkout Code') {
20      steps {
21        git branch: 'main', url: 'https://github.com/cchahid/python-app.git'
22      }
23    }
24
25    stage('SSH Command') {
26      steps {
27        script {
28          sh "ssh -i /var/lib/jenkins/.ssh/
29            jenkins_rsa -o StrictHostKeyChecking=no
30            azureuser@172.201.196.66 echo Hello World
31          "
32        }
33      }
34    }
35  }
```

```
1      stage('Install Dependencies') {
2          steps {
3              dir('src') {
4                  script {
5                      sh '''
6                          set -e
7                          sudo apt-get update
8                          python3.12 -m venv myenv
9                          . myenv/bin/activate
10                         pip install -r requirements.txt
11                         '''
12                  }
13              }
14          }
15      }
16
17      stage('Run Unit Tests') {
18          steps {
19              script {
20                  sh '''
21                      set -e
22                      cd src
23                      . myenv/bin/activate
24                      pip install --upgrade pytest
25                      pytest
26                      '''
27              }
28          }
29      }
30
31      stage('Build Docker Image') {
32          steps {
33              script {
34                  sh '''
35                      set -e
36                      TAG="latest.${BUILD_NUMBER}"
37                      docker build --no-cache -t
38                      ${DOCKER_REGISTRY}/${IMAGE_NAME}:${TAG}
39                      -f build/Dockerfile .
40                      '''
41              }
42          }
43      }
44  }
```

```
1      stage("Push Docker Image to Docker Hub") {
2          steps {
3              script {
4                  withCredentials([usernamePassword(credentialsId:
5                      'docker-hub-credentials', usernameVariable:
6                      'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
7                      sh '''
8                          echo "$DOCKER_PASS" | docker login -u
9                          "$DOCKER_USER" --password-stdin
10                         TAG="latest.${BUILD_NUMBER}"
11                         docker push docker.io/cchahi/python-app:$TAG
12                         '''
13                  }
14              }
15          }
16      }
17
18      stage("Deploy to Azure Kubernetes") {
19          steps {
20              script {
21                  sh '''
22                      sed -i "s|image: ${DOCKER_REGISTRY}/
23                      ${IMAGE_NAME}:.*|image: ${DOCKER_REGISTRY}/
24                      ${IMAGE_NAME}:latest.${BUILD_NUMBER}|g"
25                      ${K8S_MANIFEST_PATH}/python-app-deployment.yaml
26
27                      export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
28
29                      kubectl config view
30
31                      kubectl apply -f ${K8S_MANIFEST_PATH}/python-app-
32                      deployment.yaml
33                      kubectl apply -f ${K8S_MANIFEST_PATH}/python-app-
34                      service.yaml
35                      '''
36              }
37          }
38      }
39
40      post {
41          always {
42              sh 'docker system prune -f'
43          }
44      }
45  }
46 }
```

Explication du Jenkinsfile

1. Agent

L'agent `any` indique que le pipeline peut être exécuté sur n'importe quel agent disponible dans l'environnement Jenkins. Cela permet une flexibilité dans le choix du serveur pour l'exécution des tâches du pipeline.

2. Environnement

La section `environment` définit les variables d'environnement qui seront utilisées tout au long du pipeline :

- `KUBECONFIG` : Chemin vers le fichier de configuration Kubernetes.
- `IMAGE_NAME` : Nom de l'image Docker à construire.
- `CONTAINER_PORT` et `HOST_PORT` : Ports utilisés par l'application dans le conteneur et sur l'hôte.
- `DOCKER_REGISTRY` : Le registre Docker Hub où l'image sera poussée.
- `DOCKER_TAG` : Le tag à utiliser pour l'image Docker, basé sur le numéro de build Jenkins.
- `ANSIBLE_PLAYBOOK` : Fichier Ansible pour le déploiement (non utilisé ici mais mentionné pour référence).
- `SSH_KEY` : Chemin vers la clé SSH pour l'accès à distance (non utilisé dans le pipeline, mais peut être utile dans d'autres étapes).
- `K8S_MANIFEST_PATH` : Chemin vers les fichiers de configuration Kubernetes.
- `DEPLOYMENT_NAME` et `CONTAINER_NAME` : Nom du déploiement et du conteneur.

3. Stage 'Checkout Code'

Cette étape extrait le code source du dépôt GitHub spécifié (branche `main`) pour l'utiliser dans le pipeline. Elle utilise la commande `git` pour cloner le dépôt et s'assurer que le code de la dernière version est disponible.

4. Stage 'SSH Command'

Cette étape utilise une connexion SSH pour exécuter une commande sur un serveur distant. Ici, il s'agit simplement d'une commande qui renvoie "Hello World" pour tester la connexion SSH. Vous utilisez la clé privée `jenkins_rsa` pour vous connecter à l'IP `172.201.196.66` avec l'utilisateur `azureuser`.

5. Stage 'Install Dependencies'

Dans cette étape, les dépendances Python nécessaires à l'application sont installées :

- Mise à jour de la liste des paquets avec `sudo apt-get update`.
- Création d'un environnement virtuel Python (`venv`) nommé `myenv`.
- Activation de l'environnement virtuel et installation des dépendances à partir du fichier `requirements.txt`.

6. Stage 'Run Unit Tests'

Exécute les tests unitaires pour s'assurer que le code fonctionne comme prévu :

- Active l'environnement virtuel.
- Met à jour `pytest` et exécute les tests avec `pytest`.

7. Stage 'Build Docker Image'

Cette étape crée une image Docker à partir du code source et des fichiers de configuration spécifiés :

- Utilise le `Dockerfile` situé dans le répertoire `build/` pour construire l'image.
- L'image est taguée avec la version `latest.BUILD_NUMBER,o`

8. Stage 'Push Docker Image to Docker Hub'

Cette étape pousse l'image Docker construite précédemment vers Docker Hub :

- Utilise les identifiants Docker Hub fournis par Jenkins via `withCredentials` pour se connecter à Docker Hub.
- Connecte Docker avec les identifiants et pousse l'image avec le tag généré.

9. Stage 'Deploy to Azure Kubernetes'

Déploie l'application sur un cluster Kubernetes Azure en mettant à jour les fichiers de manifeste Kubernetes et en appliquant les changements :

- Utilise `sed` pour mettre à jour le manifeste Kubernetes (`python-app-deployment.yaml`) avec le tag de l'image Docker généré.
- Configure `kubectl` pour pointer vers le fichier de configuration Kubernetes (`/etc/rancher/k3s/k3s.yaml`).
- Applique les manifestes Kubernetes pour déployer l'application et exposer le service via `kubectl apply`.

10. Section 'Post'

Dans cette section, après l'exécution des étapes, la commande `docker system prune -f` est exécutée pour nettoyer l'environnement Docker, supprimant les images inutilisées et libérant de l'espace disque.

Conclusion

Ce pipeline est conçu pour automatiser le processus de développement, test, création d'image Docker, et déploiement sur un cluster Kubernetes. Chaque étape assure un contrôle de qualité (tests unitaires) et une gestion fluide du cycle de vie de l'application, depuis le développement jusqu'à son déploiement sur Kubernetes.

3.6 Tester le pipeline

Pour tester le pipeline, on doit faire des changements sur notre application et ajouter ces changements dans la branche spécifiée pour déclencher le pipeline (main dans notre cas). On peut voir après que le pipeline commence un build et si le pipeline a été réussi, on peut voir en accédant à la vue générale du pipeline comme l'image ci-dessous.

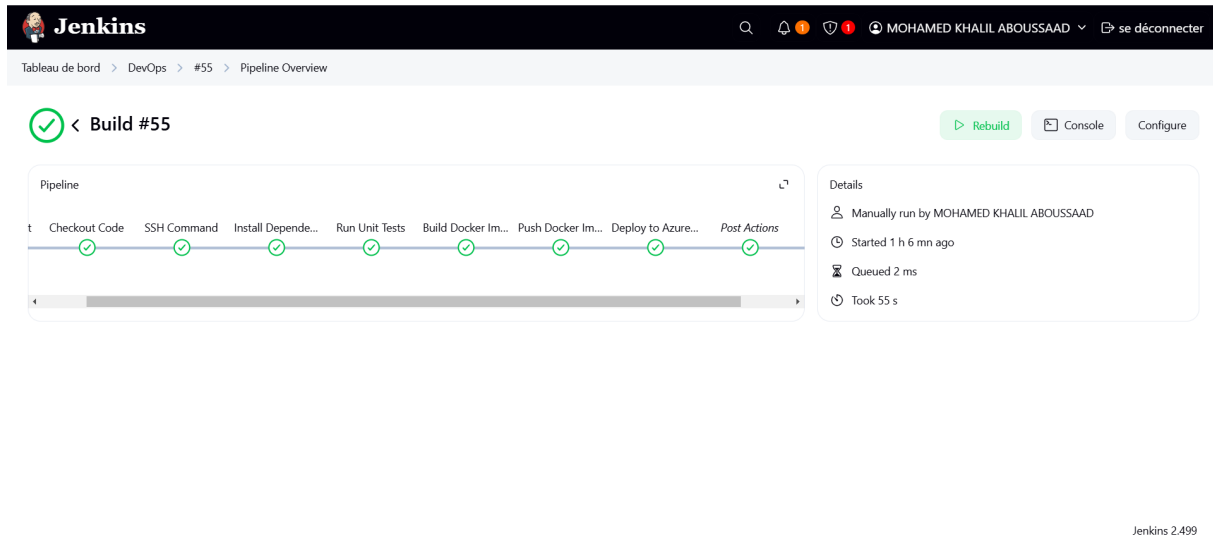


FIGURE 3.8 – Vue d’ensemble du pipeline après déclenchement d’un build

3.7 Vérifier l'état et accéder à l'application

Pour vérifier l'état de l'application, on peut accéder à la machine virtuelle en utilisant SSH.

```
khalil@DESKTOP-MRAFST3:~$ ssh azureuser@172.201.196.66
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 5.4.0-1109-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Feb 27 20:42:08 UTC 2025

System load:  0.14           Users logged in:      0
Usage of /:   22.0% of 28.89GB IP address for eth0:   10.0.1.6
Memory usage: 34%           IP address for docker0: 172.17.0.1
Swap usage:   0%            IP address for cni0:   10.42.0.1
Processes:   137

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Infrastructure is not enabled.

0 updates can be applied immediately.

167 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 18.04 at
https://ubuntu.com/18-04

New release '20.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Feb 27 16:49:25 2025 from 105.188.42.146
azureuser@masterVM:~$ █
```

FIGURE 3.9 – Accès à la machine virtuelle via SSH

Utiliser la commande pour savoir l'état des nœuds :

```
azureuser@masterVM:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
mastervm	Ready	control-plane,master	2d4h	v1.31.5+k3s1
worker1vm	Ready	<none>	2d4h	v1.31.5+k3s1

FIGURE 3.10 – État des nœuds Kubernetes

Voir l'état et l'âge des pods, ils doivent être récents :

```
azureuser@masterVM:~$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	python-app-589db87477-d7h66	1/1	Running	0	70m
default	test-pod	1/1	Running	0	2d4h
kube-system	coredns-ccb96694c-qcxgj	1/1	Running	0	2d4h
kube-system	helm-install-traefik-crd-gf29q	0/1	Completed	0	2d4h
kube-system	helm-install-traefik-mnshl	0/1	Completed	1	2d4h
kube-system	local-path-provisioner-5cf85fd84d-fmzgl	1/1	Running	0	2d4h
kube-system	metrics-server-5985cbc9d7-qwtgx	1/1	Running	0	2d4h
kube-system	svclb-python-app-service-9d486cc6-4nqch	1/1	Running	0	18h
kube-system	svclb-python-app-service-9d486cc6-wt8kc	1/1	Running	0	18h
kube-system	svclb-traefik-024a276a-g2r4n	2/2	Running	0	2d4h
kube-system	svclb-traefik-024a276a-zqjp4	2/2	Running	0	2d4h
kube-system	traefik-5d45fc8cc9-mhd6l	1/1	Running	0	2d4h

FIGURE 3.11 – État et âge des pods Kubernetes

On teste l'accès à l'application depuis l'adresse IP du worker et le port 32694 :

```
azureuser@masterVM:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.43.0.1	<none>	443/TCP	2d4h
python-app-service	LoadBalancer	10.43.208.184	10.0.1.5,10.0.1.6	5000:32694/TCP	18h
test-pod	NodePort	10.43.115.150	<none>	80:30908/TCP	2d4h

FIGURE 3.12 – Accès à l'application via l'adresse IP du worker et le port 32694